

DATA-260 Homework 1 – Report

Uday Patel

Section 0 – Personal Configuration and Domain

Value	Result
SID4	9871
PORT_BASE	8871
PREFIX	s9871
SEED	9871
VERIFY_SEED	269871
DOMAIN_ID	7 – Community sports league fixtures

Hardware: Lenovo Legion Slim 5 16” (Ryzen 5 7640HS, 16GB RAM, NVIDIA RTX 4060 8GB, 512GB SSD)

Local model used: qwen3:8b, served via Ollama

Repository: <https://github.com/UdayPate/data260-9871>

Tagged commit: hw1 (see repository tags)

Part 1 – HTML, JavaScript, Docker, AWS ECS

A Community Sports League Fixtures form (`index.html`) was built with a required primary field (fixture name, autofocus), secondary field (teams/players), submitter email, description textarea, sport-type dropdown, and a terms-and-conditions checkbox. `app.js` implements arrow-function validation (description >25 characters, checkbox checked), `JSON.stringify/JSON.parse` of the form data, object destructuring of the fixture name and email, the spread operator to add a `submissionDate` field, and a closure-based submission counter. All behaviors were verified in the browser console (validation alerts, JSON logging, destructuring, spread, and an incrementing submission count).

Browser testing

The form was tested directly in the browser with the DevTools console open, confirming the JSON string produced by `JSON.stringify`, the destructured `fixtureName` and `submitterEmail` values, and the updated object produced by the spread operator (including the added `submissionDate` field):

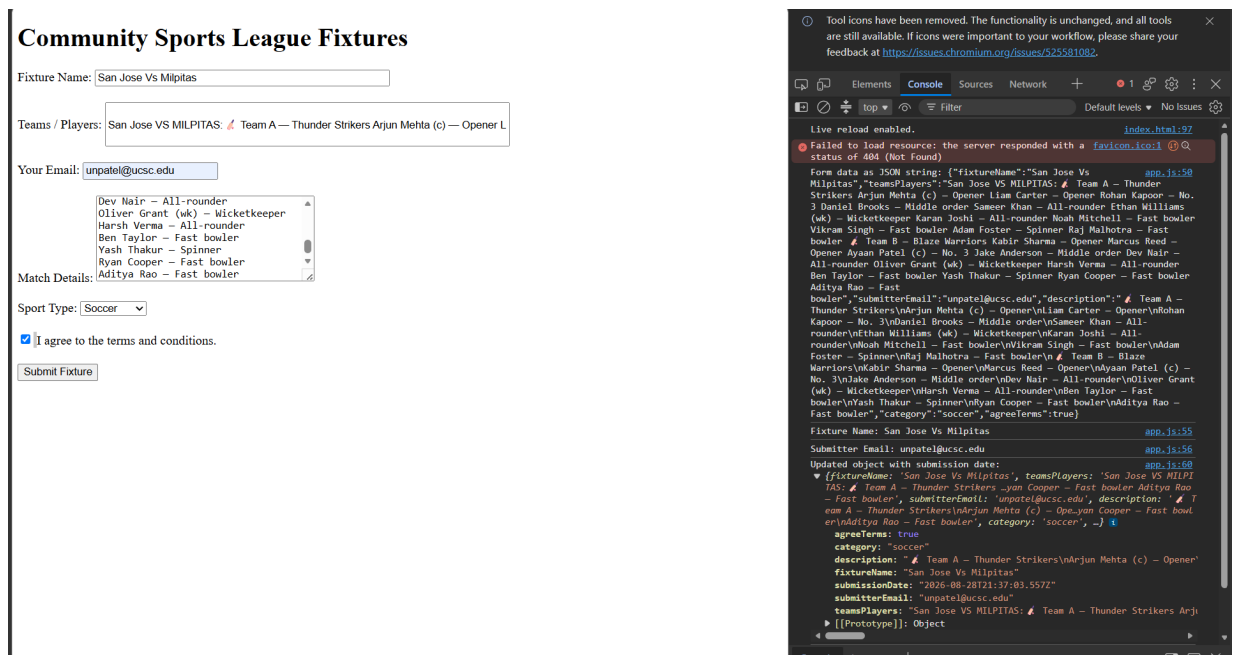


Figure 1: Form filled out in the browser, with the console showing the JSON-stringified form data, destructured `fixtureName/submitterEmail`, and the spread-operator updated object with `submissionDate` added. The closure-based submission counter was also verified separately by submitting the form multiple times and observing the logged count increment each time.

Docker deployment

The app was containerized with an Nginx-based Dockerfile serving on PORT_BASE (8871), built and run locally, and verified in the browser at `localhost:8871`:

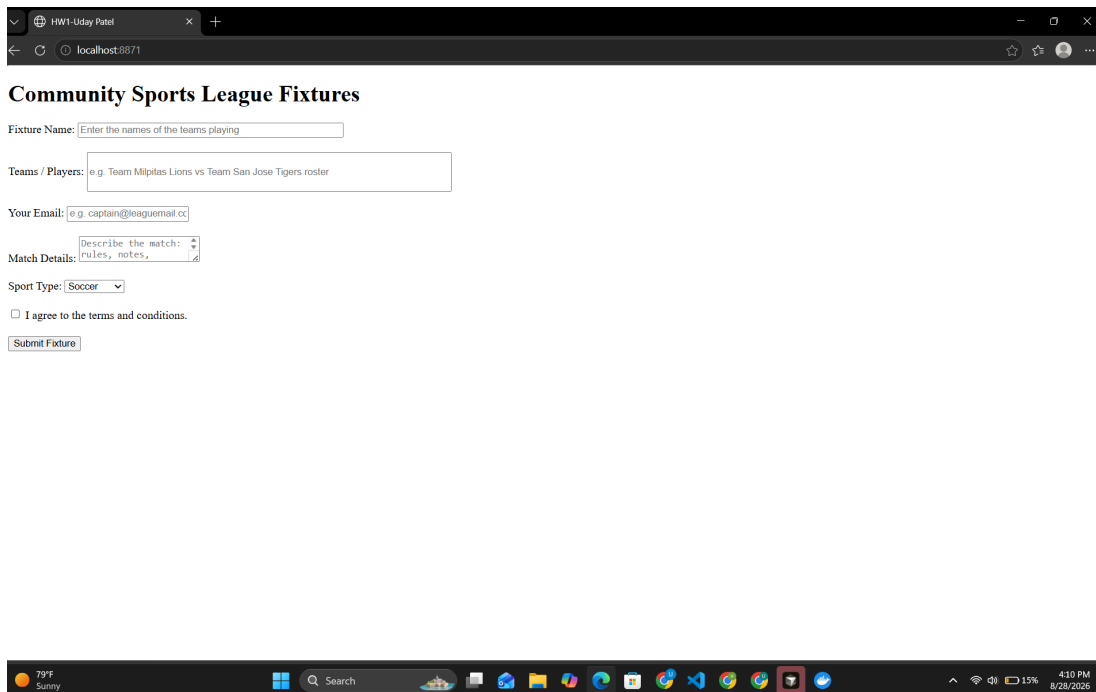


Figure 2: Application running locally via Docker (localhost:8871).

AWS ECS deployment

The image was pushed to a private Amazon ECR repository (`my_web_app`), then deployed as a single Fargate task (0.25 vCPU / 0.5 GB) in an ECS service, with a dedicated security group allowing inbound TCP traffic on port 8871 from any source, and a public IP enabled. The running task was verified at its public IP:

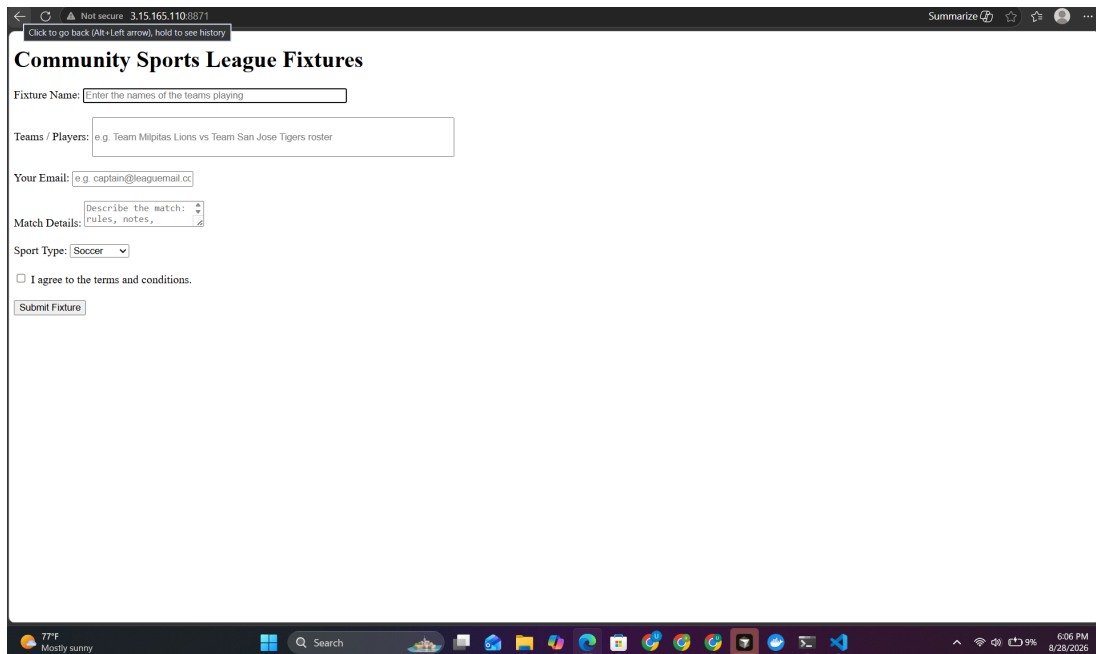


Figure 3: Application running on AWS ECS/Fargate, accessed at its public IP (3.15.165.110:8871). The IP address is still up and running and you can check it.

Part 2 – Agentic AI (Planner / Reviewer / Finalizer)

Two agents (Planner, Reviewer) plus a Finalizer step were built in `agents_demo.py`, using qwen3:8b via Ollama and LangChain. The Planner proposes 3 tags and a ≤ 25 -word summary purely from the input title/content (no hardcoded domain keywords). The Reviewer checks the draft against the same two criteria and either confirms it or corrects it. The Finalizer strips any model reasoning trace and parses the result into strict JSON.

Command used

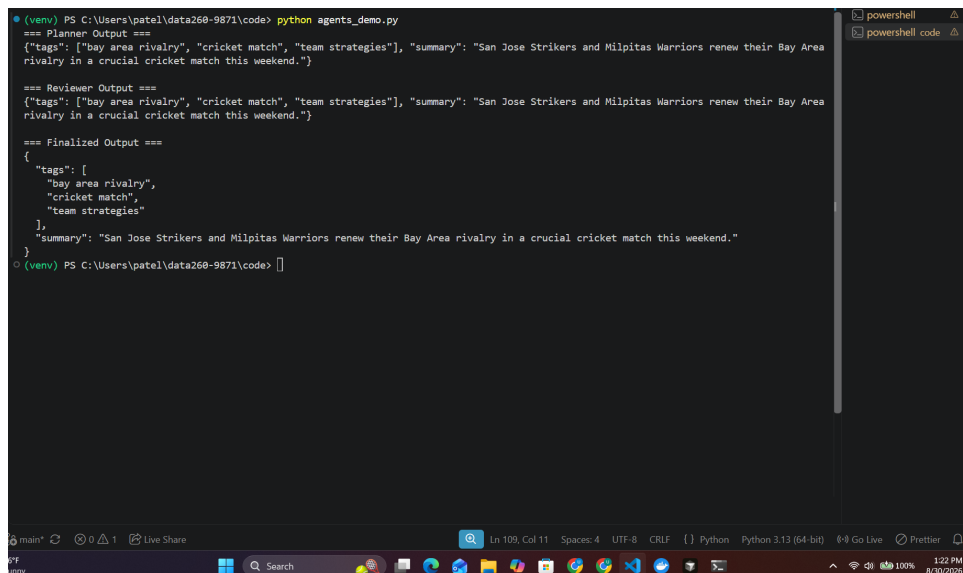
```
python agents_demo.py
```

Console output (Planner, Reviewer, Finalized)

```
=== Planner Output ===
{"tags": ["bay area rivalry", "cricket match", "team strategies"],
 "summary": "San Jose Strikers and Milpitas Warriors renew their Bay
Area rivalry in a crucial cricket match this weekend."}

=== Reviewer Output ===
{"tags": ["bay area rivalry", "cricket match", "team strategies"],
 "summary": "San Jose Strikers and Milpitas Warriors renew their Bay
Area rivalry in a crucial cricket match this weekend."}

=== Finalized Output ===
{
  "tags": [
    "bay area rivalry",
    "cricket match",
    "team strategies"
  ],
  "summary": "San Jose Strikers and Milpitas Warriors renew their Bay
Area rivalry in a crucial cricket match this weekend."
}
```



```
(venv) PS C:\Users\patel\data260-9871\code> python agents_demo.py

=== Planner Output ===
{"tags": ["bay area rivalry", "cricket match", "team strategies"], "summary": "San Jose Strikers and Milpitas Warriors renew their Bay Area rivalry in a crucial cricket match this weekend."}

=== Reviewer Output ===
{"tags": ["bay area rivalry", "cricket match", "team strategies"], "summary": "San Jose Strikers and Milpitas Warriors renew their Bay Area rivalry in a crucial cricket match this weekend."}

=== Finalized Output ===
{
  "tags": [
    "bay area rivalry",
    "cricket match",
    "team strategies"
  ],
  "summary": "San Jose Strikers and Milpitas Warriors renew their Bay Area rivalry in a crucial cricket match this weekend."
}
(venv) PS C:\Users\patel\data260-9871\code>
```

Figure 4: Terminal output of `python agents_demo.py`, showing the Planner, Reviewer, and Finalized JSON output.

Short answers

Q1. The 3 final tags: “bay area rivalry”, “cricket match”, “team strategies”

Q2. The final summary: “San Jose Strikers and Milpitas Warriors renew their Bay Area rivalry in a crucial cricket match this weekend.” (18 words)

Q3. Did the Reviewer change anything? No. The Reviewer received the Planner’s draft and returned it unchanged, since the draft already had exactly 3 specific tags and an 18-word summary that accurately reflected the input thus both conditions the Reviewer was instructed to check were already satisfied.

Explanation of each step

Planner: receives the raw title/content and is prompted to propose exactly 3 tags and a one-sentence, ≤ 25 -word summary, deriving both purely from the text rather than fixed categories.

Reviewer: receives the Planner’s draft (plus the original title/content) and checks it against the same two criteria (tag count/specificity, summary length/accuracy) – either confirming it unchanged or correcting it.

Finalizer: takes the Reviewer’s raw text output, strips any internal reasoning trace the model may have emitted (qwen3:8b sometimes wraps output in a reasoning block), extracts the JSON object, and returns a clean Python dict with exactly the **tags** and **summary** keys required by the assignment.

Part 3 – Measuring Non-Determinism

Using a fixed input (saved at `reports/hw01/cases/nondeterminism_input.json`), the full Planner-Reviewer-Finalizer pipeline was run 20 times at temperature 0.7 and 20 times at temperature 0.0, recording the final tags and latency for each run.

Command used

```
python measure_nondeterminism.py
```

Tag Set Variation

Metric	Temp 0.7	Temp 0.0
Distinct tag sets	13	1
Tags in all 20 runs	(none)	bay area rivalry, cricket match, team rivalry
Tags in exactly 1 run	bay area, fast bowling, game strategy, green pitch, league title race, middle order, strategic play, team rivalry, team standings, team strategies, top league teams	(none)

Latency

Metric	Temp 0.7	Temp 0.0
Latency p50 (ms)	49282.5	48720.5
Latency p95 (ms)	72209.5	49382.9
Latency p99 (ms)	79907.2	49721.1

Explanation

At temperature 0.0, the pipeline produced identical tags across all 20 runs, confirming qwen3:8b is effectively deterministic at this setting on this hardware. At temperature 0.7, only 13 of 20 runs produced a unique tag combination, and no single tag appeared in every run – even “bay area rivalry” and “cricket match,” which appeared in 19/20 runs, each had one outlier run. Latency was also far more consistent at temp 0.0 (a roughly 1-second range) than at temp 0.7 (a roughly 57-second range), suggesting randomness affects not just word choice but how much the model explores before settling on an answer.

Two users sending identical input at temperature 0.0 would see exactly the same tags and summary every time – useful when consistency matters, such as an automated compliance or audit classification system where the same input must always yield the same category for legal defensibility. Two users sending identical input at temperature 0.7 would likely see different (though related) tags – acceptable, and even desirable, for a creative tag-suggestion feature where variety helps users discover different angles on their content, but not acceptable for a system that needs reproducible, auditable output.

Part 4 – Model Client and Token Accounting

A reusable model-adaptor class (`ModelClient`) was built at `src/model_client.py`, exposing a stable `complete(messages, tools=None)` interface. All model calls in `hw1_client.py` go through this adaptor, which tracks conversation history and cumulative token counts internally. `hw1_client.py` is a small CLI that loads `AGENT.md` as the system prompt (instructing the model to act as a strict, bullet-only code reviewer) and supports a `/stats` command.

Command used

```
python hw1_client.py
```

5-turn conversation, with `/stats` after turn 3 and turn 5

```
You: Review this code: def divide(a, b): return a / b
[turn 1] input_tokens=127 output_tokens=384 total_tokens=511
Assistant: - No handling for division by zero (b == 0)
- No type validation for inputs
- Uses floating-point division (/) instead of integer division (//) if intended
- No error handling for non-numeric inputs

You: Review this code: def add(a, b): return a+b
[turn 2] input_tokens=294 output_tokens=606 total_tokens=900
Assistant: - No type validation for inputs (e.g., non-numeric types like strings)
- No error handling for invalid operations (e.g., adding incompatible types)
- No documentation or clarity on expected input types
- Potential silent failure if inputs are not numbers

You: What did the first function I asked you to review do?
[turn 3] input_tokens=367 output_tokens=482 total_tokens=849
Assistant: The first function you asked me to review was divide(a, b),
which performs floating-point division of a by b and returns the
result. It lacks error handling for division by zero, type
validation, and non-numeric inputs.

You: /stats
--- /stats ---
Turn count: 3
Cumulative input tokens: 788
Cumulative output tokens: 1472
Cumulative total tokens: 2260
Serialized history length (chars): 1541
-----

You: Thanks, that's helpful
[turn 4] input_tokens=434 output_tokens=270 total_tokens=704
Assistant: You're welcome! If you have more code to review or
questions, feel free to ask.

You: Can you review one more: def subtract(a, b): return a - b
[turn 5] input_tokens=482 output_tokens=379 total_tokens=861
Assistant: - No type validation for inputs (e.g., non-numeric types like strings)
- No error handling for invalid operations (e.g., subtracting incompatible types)
- No documentation or clarity on expected input types
- Potential silent failure if inputs are not numbers

You: /stats
--- /stats ---
```

```
Turn count: 5
Cumulative input tokens: 1704
Cumulative output tokens: 2121
Cumulative total tokens: 3825
Serialized history length (chars): 2123
-----

You: /exit
=== Final Summary ===
Total turns: 5
Cumulative input tokens: 1704
Cumulative output tokens: 2121
Cumulative total tokens: 3825
```

Report and README answers

Why is prior conversation context resent with every turn? LLMs are stateless between API calls – the model has no memory of earlier messages once a request finishes. The only way it can “remember” earlier turns is if the full history is explicitly included in the current request. This is demonstrated directly by turn 3 above: the model correctly recalled the exact `divide(a, b)` function from turn 1 only because that entire exchange was resent as part of turn 3’s request.

How is a system prompt different from a user message? A system prompt sets the model’s behavior, role, or constraints for the whole conversation – instructions about how to respond, not content to respond to. A user message is conversational content the model reacts to. Here, `AGENT.md` is sent as the system role on every turn, and its “bullet-only code reviewer” instruction shaped every reply even though no individual user message asked for bullets.

Why do input tokens grow over a conversation? Because the full history is resent every turn, each new request contains strictly more content than the last (system prompt + all previous turns + the new message). This is visible above: turn 1 had 127 input tokens, but by turn 5 input tokens had grown to 482 – nearly 4x – purely because turns 1–4’s messages and replies were now all carried along inside turn 5’s request.

What eventually limits that growth? The model’s context window – a fixed maximum number of tokens (input and output combined) a given model can process in a single request. Once the resent history plus the new message would exceed that limit, older messages must be truncated or summarized, or the request fails. `hw1_client.py` as written never trims history, so in a long enough conversation it would eventually hit this ceiling.